

Normalização de Dados é um processo de design de banco de dados relacional que organiza as tabelas e colunas para reduzir a redundância de dados e proteger a integridade das informações. O objetivo principal é evitar anomalias (erros ou inconsistências) na hora de inserir, atualizar ou excluir dados.

As regras são aplicadas em etapas, chamadas de Formas Normais. É um processo progressivo: para uma tabela estar na 2ª Forma Normal, ela precisa obrigatoriamente já atender às regras da 1ª Forma Normal, e assim por diante.

1ª Forma Normal (1FN): Atomicidade:

Regra: Todos os atributos (colunas) devem ser atômicos, ou seja, indivisíveis. Não pode haver grupos repetidos ou múltiplos valores em uma única célula (como listas ou arrays separadas por vírgula). Além disso, cada linha deve ser única.

Antes (Viola a 1FN): A coluna Telefones contém mais de um valor, o que dificulta buscas e atualizações.

ID_Cliente	Nome	Telefones
1	Ana	119999-0000, 118888-0000
2	Carlos	117777-0000

Depois (Na 1FN): Cada célula contém apenas um valor. (Nota: Na prática do dia a dia, para evitar a repetição do nome da Ana, criaríamos uma tabela separada só para os telefones, mas a divisão abaixo já satisfaz a regra estrita da 1 FN).

ID_Cliente	Nome	Telefone
1	Ana	119999-0000
1	Ana	118888-0000
2	Carlos	117777-0000

2ª Forma Normal (2FN): Dependência Funcional Total:

Regra: A tabela deve estar na 1FN e não possuir dependências parciais. Isso significa que, se a tabela tiver uma chave primária composta (formada por duas ou mais colunas), todos os outros atributos devem depender de toda a chave, e não apenas de uma parte dela. Se a tabela tem uma chave primária simples (uma só coluna), ela já está automaticamente na 2FN.

Antes (Viola a 2FN): Chave Primária Composta: ID_Pedido + ID_Produto

ID_Pedido	ID_Produto	Nome_Produto	Quantidade
100	50	Teclado	2
100	51	Mouse	1

O problema: A coluna Quantidade depende de qual pedido e qual produto estamos falando (depende de toda a chave). Porém, o Nome_Produto depende apenas do ID_Produto. Isso causa repetição de dados desnecessária.

Depois (Na 2FN): Separamos os dados em duas tabelas para eliminar a redundância.

Tabela: Itens_Pedido

ID_Pedido	ID_Produto	Quantidade
100	50	2
100	51	1

Tabela: Produtos:

ID_Produto	Nome_Produto
50	Teclado
51	Mouse

3ª Forma Normal (3FN): Sem Dependências Transitivas

Regra: A tabela deve estar na 2FN e não pode conter dependências transitivas. Isso significa que um atributo que não é chave primária não pode depender de outro atributo que também não é chave. Uma frase famosa na área de banco de dados resume a 3FN: "Tudo deve depender da chave, de toda a chave, e de nada mais além da chave".

Antes (Viola a 3FN): Chave Primária: ID_Funcionario

ID_Funcionario	Nome	ID_Departamento	Nome_Departamento
1	Ana	10	Tecnologia
1	Carlos	10	Tecnologia

O problema: O Nome_Departamento depende do ID_Departamento, que por sua vez depende do ID_Funcionario. Se o nome do departamento mudar de "Tecnologia" para "TI", teríamos que atualizar múltiplas linhas, correndo o risco de inconsistência.

Depois (Na 3FN): Movemos o que é dependente para uma tabela própria, deixando apenas a chave estrangeira (ID_Departamento) na tabela original.

Tabela: Funcionarios

ID_Funcionario	Nome	ID_Departamento
1	Ana	10
1	Carlos	10